

Dokumentacja implementacyjna

System wizualizacji grafów planarnych

Projekt w języku Java (Swing)

Autorzy:

Krzysztof Wasilewski, Jakub Pietrzkiewicz

Data: 19.05.2026

Spis treści

1	Wstęp	3
1.1	Cel dokumentu	3
1.2	Zakres	3
2	Architektura systemu	4
2.1	Podział na moduły	4
2.2	Przepływ danych	4
3	Model danych	5
3.1	Struktury podstawowe	5
3.2	Struktury pomocnicze	5
4	Interfejsy modułów	7
4.1	Moduł Graph	7
4.2	Moduł Fruchterman	7
4.3	Moduł Tutte	7
4.4	Moduł AdjacencyList	7
4.5	Kody statusu	7
5	Specyfikacja danych wejściowych i wyjściowych	9
5.1	Format wejściowy (tekst)	9
5.2	Format wejściowy (binarny)	9
5.3	Format wyjściowy (tekst)	9
5.4	Format wyjściowy (binarny)	10
6	Implementacja algorytmów	11
6.1	Fruchterman-Reingold	11
6.2	Tutte	12
7	Obsługa błędów i sytuacje wyjątkowe	15
8	Kompilacja i uruchamianie	16
8.1	Wymagania	16
8.2	Kompilacja	16
8.3	Uruchamianie GUI (start z Main)	16

1 Wstęp

1.1 Cel dokumentu

Celem dokumentacji implementacyjnej jest przedstawienie szczegółów budowy aplikacji GUI w języku Java (Swing) do wyznaczania współrzędnych węzłów grafu planarnego. Dokument opisuje strukturę kodu, model danych, formaty wejścia i wyjścia oraz obsługę błędów na podstawie aktualnej implementacji, z naciskiem na warstwę obliczeniową i jej integrację z GUI.

1.2 Zakres

Dokument obejmuje:

- architekturę modułową części obliczeniowej,
- model danych i struktury pomocnicze,
- implementację algorytmów Fruchterman-Reingold i Tutte,
- specyfikację wejścia i wyjścia (tekst i binaria) zgodną z formatem z części C,
- kody statusu oraz scenariusze błędowe,
- sposób budowania projektu Maven.

2 Architektura systemu

2.1 Podział na moduły

Implementacja znajduje się w pakiecie `pl.graph` i składa się z następujących klas:

- `Graph` - odczyt i zapis grafu oraz mapowanie identyfikatorów węzłów,
- `Node` - model węzła z identyfikatorem i współrzędnymi,
- `Edge` - model krawędzi (indeksy w tablicy węzłów, waga, etykieta),
- `AdjacencyList` - budowa list sąsiedztwa dla algorytmu Tutte,
- `Neighbor` - pojedynczy element listy sąsiadów,
- `Fruchterman` - algorytm siłowy wyznaczania układu,
- `Tutte` - osadzenie Tutte na podstawie zredukowanej macierzy Laplace'a,
- `Gui` - moduł interfejsu graficznego (Swing),
- `ExitCodes` - wspólne kody statusu dla warstwy obliczeniowej.

Moduł GUI oparty o Swing odpowiada za warstwę prezentacji, obsługę zdarzeń i sterowanie wywołaniami algorytmów.

2.2 Przepływ danych

Warstwa obliczeniowa działa w trzech etapach:

1. Wczytanie grafu do struktury `Graph` (tekst lub binaria zgodne z częścią C).
2. Uruchomienie wybranego algorytmu rozmieszczenia węzłów.
3. Zapis współrzędnych w formacie tekstowym lub binarnym.

Każdy etap jest niezależny i może zostać wywołany przez warstwę sterującą (GUI).

3 Model danych

3.1 Struktury podstawowe

W module Graph zdefiniowane są trzy podstawowe modele:

- Node
 - id - identyfikator logiczny węzła,
 - x, y - współrzędne geometryczne.
- Edge
 - firstNodeIndex, secondNodeIndex - indeksy w tablicy nodes,
 - weight - waga krawędzi,
 - label - etykieta (maks. 32 znaki, nadmiar obcinany).
- Graph
 - nodes - tablica węzłów,
 - nodesCount - liczba węzłów,
 - edges - tablica krawędzi,
 - edgesCount - liczba krawędzi.

Węzły są sortowane po id, a mapowanie identyfikatorów do indeksów realizowane jest przez słownik nodeIdToIndex.

Fragment implementacji klasy Node:

```
public class Node implements Comparable<Node> {
    public int id;
    public double x;
    public double y;

    public Node(int id) {
        this.id = id;
        this.x = 0.0;
        this.y = 0.0;
    }

    @Override
    public int compareTo(Node other) {
        return Integer.compare(this.id, other.id);
    }
}
```

Fragment implementacji klasy Edge:

```
public Edge(int firstNodeIndex, int secondNodeIndex, double weight, String label) {
    this.firstNodeIndex = firstNodeIndex;
    this.secondNodeIndex = secondNodeIndex;
    this.weight = weight;
    this.label = label != null ? label : "";
}
```

3.2 Struktury pomocnicze

- AdjacencyList - tablica list sąsiedztwa i stopni węzłów,
- Neighbor - element listy sąsiadów: indeks węzła, waga, wskaźnik next.

Fragment implementacji budowy listy sąsiedztwa:

```
for (int i = 0; i < graph.edgesCount; i++) {
    int firstIndex = graph.edges[i].firstNodeIndex;
    int secondIndex = graph.edges[i].secondNodeIndex;
    double weight = graph.edges[i].weight;

    Neighbor neighbor1 = new Neighbor(secondIndex, weight);
    neighbor1.next = adjList.adjacencyList[firstIndex];
    adjList.adjacencyList[firstIndex] = neighbor1;

    Neighbor neighbor2 = new Neighbor(firstIndex, weight);
    neighbor2.next = adjList.adjacencyList[secondIndex];
    adjList.adjacencyList[secondIndex] = neighbor2;

    adjList.degrees[firstIndex]++;
    adjList.degrees[secondIndex]++;
}
```

4 Interfejsy modułów

4.1 Moduł Graph

Najważniejsze metody publiczne:

- Graph loadGraph(String path)
- ExitCodes saveGraphAsText(Graph graph, String path)
- ExitCodes saveGraphAsBinary(Graph graph, String path)
- int getNodeIndex(Graph graph, int nodeId)

Fragment mapowania identyfikatora na indeks (wyszukiwanie binarne):

```
public static int getNodeIndex(Graph graph, int nodeId) {
    Node key = new Node(nodeId);
    int result = Arrays.binarySearch(graph.nodes, key);
    return result < 0 ? -1 : result;
}
```

Fragment parsowania wejścia i obcinania etykiety:

```
String label = parts[0];
if (label.length() > 32) {
    label = label.substring(0, 32);
}

int firstNode = Integer.parseInt(parts[1]);
int secondNode = Integer.parseInt(parts[2]);
if (firstNode < 0 || secondNode < 0) {
    continue;
}
```

Fragment mapowania krawędzi na indeksy węzłów:

```
Integer firstIndex = nodeIdToIndex.get(edge.firstNodeId);
Integer secondIndex = nodeIdToIndex.get(edge.secondNodeId);
if (firstIndex == null || secondIndex == null) {
    throw new IllegalArgumentException("Error: Node ID not found");
}
graph.edges[i] = new Edge(firstIndex, secondIndex, edge.weight, edge.label);
```

4.2 Moduł Fruchterman

- konstruktor Fruchterman(double size, int iterations, Graph graph)
- void layout(Graph graph)

4.3 Moduł Tutte

- ExitCodes runTutte(Graph graph, double size, int maxIterations)

4.4 Moduł AdjacencyList

- AdjacencyList createAdjacencyList(Graph graph)

4.5 Kody statusu

Wspólne kody statusu zdefiniowane są w ExitCodes. Najczęściej używane w warstwie obliczeniowej:

- SUCCESS

- FILE_ERROR
- OUTPUT_WRITE_ERROR
- TUTTE_ASSUMPTIONS_ERROR
- NUMERICAL_ERROR
- EMPTY_OR_INVALID_GRAPH

5 Specyfikacja danych wejściowych i wyjściowych

5.1 Format wejściowy (tekst)

Plik wejściowy jest listą krawędzi:

```
<label> <nodeA> <nodeB> <weight>
```

Reguły parsowania:

- linie puste i komentarze # są pomijane,
- komentarze inline po danych są odcinane,
- label jest skracany do 32 znaków,
- nodeA, nodeB muszą być liczbami całkowitymi nieujemnymi,
- weight jest opcjonalny, domyślnie 1.0,

Fragment filtrowania komentarzy i pustych linii:

```
line = line.trim();
if (line.isEmpty() || line.startsWith("#")) {
    continue;
}
int commentIndex = line.indexOf('#');
if (commentIndex >= 0) {
    line = line.substring(0, commentIndex).trim();
}
if (line.isEmpty()) {
    continue;
}
```

5.2 Format wejściowy (binarny)

Zgodny z formatem z części C i dokumentacją funkcjonalną:

1. uint32_t / int: liczba wierzchołków N .
2. Ciąg N rekordów:
 - uint32_t / int: ID wierzchołka,
 - double: współrzędna X,
 - double: współrzędna Y.

Kolejność pól w rekordzie jest stała. W implementacji Java zapisy i odczyty są wykonywane jawnie w kolejności little-endian, aby zachować zgodność z eksportem z modułu C.

5.3 Format wyjściowy (tekst)

Każdy rekord opisuje węzeł:

```
<node_id> <x> <y>
```

Przykład zapisu:

```
writer.printf(Locale.ROOT, "%d %.6f %.6f\n", node.id, node.x, node.y);
```

Obsługa błędów zapisu:

```
try (PrintWriter writer = new PrintWriter(new FileWriter(path))) {
    // zapis
    return ExitCodes.SUCCESS;
} catch (IOException e) {
```

```

        System.err.println("Error: Cannot write to file: " + path);
        return ExitCodes.OUTPUT_WRITE_ERROR;
    }
}

```

5.4 Format wyjściowy (binarny)

Plik binarny zapisuje:

1. nodesCount jako 32-bitowy int w kolejności little-endian,
2. dla każdego węzła: id (int), x (double), y (double) w kolejności little-endian.

Fragment implementacji zapisu:

```

dos.writeInt(Integer.reverseBytes(graph.nodesCount));
for (int i = 0; i < graph.nodesCount; i++) {
    dos.writeInt(Integer.reverseBytes(graph.nodes[i].id));
    dos.writeLong(Long.reverseBytes(Double.doubleToLongBits(graph.nodes[i].x)));
    dos.writeLong(Long.reverseBytes(Double.doubleToLongBits(graph.nodes[i].y)));
}

```

Fragment obsługi błędu zapisu binarnego:

```

try (DataOutputStream dos = new DataOutputStream(
    new BufferedOutputStream(new FileOutputStream(path)))) {
    // zapis binarny
    return ExitCodes.SUCCESS;
} catch (IOException e) {
    System.err.println("Error: Cannot write to file: " + path);
    return ExitCodes.OUTPUT_WRITE_ERROR;
}

```

6 Implementacja algorytmów

6.1 Fruchterman-Reingold

Algorytm używa modelu siłowego z ograniczeniem do kwadratu o boku size:

- siły odpychania między każdą parą węzłów,
- siły przyciągania dla węzłów połączonych krawędzią,
- ograniczenie długości kroku przez temperaturę.

Wzory użyte w kodzie:

- $k = \sqrt{\frac{\text{area}}{|V|}}$, gdzie $\text{area} = \text{size} \cdot \text{size}$
- $f_r = \frac{k^2}{d}$
- $f_a = w \cdot \frac{d^2}{k}$

W trakcie każdej iteracji:

1. Zerowanie wektorów przemieszczeń.
2. Akumulacja sił odpychania dla każdej pary węzłów.
3. Akumulacja sił przyciągania dla każdej krawędzi.
4. Aktualizacja położenia z ograniczeniem przez temperaturę.
5. Chłodzenie temperatury liniowo do zera.

Fragment aktualizacji położenia:

```
if (distance > 0) {
    double limitedDistance = Math.min(distance, temperature);
    node.x += (dx / distance) * limitedDistance;
    node.y += (dy / distance) * limitedDistance;
}
node.x = Math.min(size, Math.max(0, node.x));
node.y = Math.min(size, Math.max(0, node.y));
```

Fragment obliczania siły odpychania między parami węzłów:

```
double dx = a.x - b.x;
double dy = a.y - b.y;
double distance = Math.hypot(dx, dy);
if (distance < 0.01) {
    dx = (random.nextDouble() - 0.5) * 0.1;
    dy = (random.nextDouble() - 0.5) * 0.1;
    distance = Math.hypot(dx, dy);
}
double force = (k * k) / distance;
double deltaX = (dx / distance) * force;
double deltaY = (dy / distance) * force;
```

Fragment uwzględniający wagę krawędzi w przyciąganiu:

```
double force = (distance * distance) / k * edge.weight;
double deltaX = (dx / distance) * force;
double deltaY = (dy / distance) * force;
```

Fragment chłodzenia temperatury:

```
double cooled = initialTemperature * (1.0 - (double) iteration / iterations);
temperature = Math.max(0.0, cooled);
```

Złożoność czasowa: $O(|V|^2 + |E|)$ na iterację.

6.2 Tutte

Algorytm przebiega etapami:

1. Dla grafów z mniej niż 4 węzłami stosowane są pozycje specjalne.
2. Tworzona jest lista sąsiedztwa.
3. Wyznaczany jest cykl brzegowy (heurystyka + fallback).
4. Węzły brzegowe rozmieszczane są równomiernie na okręgu jednostkowym.
5. Dla węzłów wewnętrznych budowana jest zredukowana macierz Laplace'a.
6. Układ równań rozwiązywany jest eliminacją Gaussa z częściowym wyborem.
7. Wynik jest skalowany do obszaru o boku size.

Fragment wczesnego wyjścia dla bardzo małych grafów:

```
if (graph.nodesCount < 4) {
    placeSmallGraph(graph);
    scaleGraph(graph, size);
    return ExitCodes.SUCCESS;
}
```

Fragment budowy listy sąsiadów i walidacji cyklu brzegowego:

```
AdjacencyList adjList = AdjacencyList.createAdjacencyList(graph);
int[] boundary = findBoundaryCycle(adjList);
if (boundary == null || boundary.length < 3) {
    return ExitCodes.TUTTE_ASSUMPTIONS_ERROR;
}
```

Fragment wyznaczania indeksów wewnętrznych wierzchołków:

```
boolean[] isBoundary = new boolean[graph.nodesCount];
for (int idx : boundary) {
    isBoundary[idx] = true;
}

int[] internalIndex = new int[graph.nodesCount];
Arrays.fill(internalIndex, -1);
int internalCount = 0;
for (int i = 0; i < graph.nodesCount; i++) {
    if (!isBoundary[i]) {
        internalIndex[i] = internalCount++;
    }
}
```

Fragment budowy zredukowanej macierzy Laplace'a i wektorów wyrazów wolnych:

```
double[][] A = new double[internalCount][internalCount];
double[] bx = new double[internalCount];
double[] by = new double[internalCount];

for (int v = 0; v < graph.nodesCount; v++) {
    if (isBoundary[v]) {
        continue;
    }
    int row = internalIndex[v];
    int degree = 0;
    for (Neighbor neighbor = adjList.adjacencyList[v]; neighbor != null; neighbor =
```

```

neighbor.next) {
    int u = neighbor.nodeIndex;
    degree++;
    if (isBoundary[u]) {
        bx[row] += graph.nodes[u].x;
        by[row] += graph.nodes[u].y;
    } else {
        int col = internalIndex[u];
        if (col >= 0) {
            A[row][col] -= 1.0;
        }
    }
}
if (degree <= 0) {
    return ExitCodes.TUTTE_ASSUMPTIONS_ERROR;
}
A[row][row] += degree;
}

```

Heurystyka cyklu brzegowego:

- start w węźle o najmniejszym stopniu,
- przejście do sąsiada minimalizującego liczbę wspólnych sąsiadów,
- w razie porażki: wybór 4 węzłów o największych stopniach.

Złożoność budowy macierzy: $O(|V| + |E|)$, złożoność eliminacji Gaussa: $O(n^3)$ dla n węzłów wewnętrznych.

Fragment rozmieszczenia węzłów brzegowych na okręgu jednostkowym:

```

for (int i = 0; i < boundaryLen; i++) {
    double angle = 2.0 * Math.PI * i / boundaryLen;
    int v = boundary[i];
    graph.nodes[v].x = Math.cos(angle);
    graph.nodes[v].y = Math.sin(angle);
}

```

Fragment przypisania rozwiązania do wierzchołków wewnętrznych:

```

double[] x = gaussianElimination(deepCopy(A), bx);
double[] y = gaussianElimination(deepCopy(A), by);
if (x == null || y == null) {
    return ExitCodes.NUMERICAL_ERROR;
}
for (int v = 0; v < graph.nodesCount; v++) {
    int row = internalIndex[v];
    if (row >= 0) {
        graph.nodes[v].x = x[row];
        graph.nodes[v].y = y[row];
    }
}

```

Fragment skalowania wyniku do zadanego rozmiaru:

```

double scale = size / 2.0;
for (int i = 0; i < graph.nodesCount; i++) {
    graph.nodes[i].x *= scale;
    graph.nodes[i].y *= scale;
}

```

Fragment eliminacji Gaussa z częściowym wyborem elementu podstawowego:

```
for (int k = 0; k < n; k++) {
    int pivot = k;
    for (int i = k + 1; i < n; i++) {
        if (Math.abs(A[i][k]) > Math.abs(A[pivot][k])) {
            pivot = i;
        }
    }
    if (Math.abs(A[pivot][k]) < 1e-12) {
        return null;
    }
    if (pivot != k) {
        double[] tmpRow = A[k];
        A[k] = A[pivot];
        A[pivot] = tmpRow;

        double tmpB = b[k];
        b[k] = b[pivot];
        b[pivot] = tmpB;
    }
    for (int i = k + 1; i < n; i++) {
        double factor = A[i][k] / A[k][k];
        A[i][k] = 0.0;
        for (int j = k + 1; j < n; j++) {
            A[i][j] -= factor * A[k][j];
        }
        b[i] -= factor * b[k];
    }
}
```

7 Obsługa błędów i sytuacje wyjątkowe

Najważniejsze scenariusze błędowe:

- brak węzłów lub krawędzi podczas wczytania grafu (wyjątek `IllegalArgumentException`),
- nieprawidłowy format danych w pliku wejściowym (linia jest ignorowana),
- brak cyklu brzegowego w Tutte (`TUTTE_ASSUMPTIONS_ERROR`),
- osobliwość układu równań w Tutte (`NUMERICAL_ERROR`),
- błędy zapisu plików (`OUTPUT_WRITE_ERROR`).
- niezgodny format binarny (niepełny plik lub błędny nagłówek) - błąd I/O lub walidacji.

8 Kompilacja i uruchamianie

8.1 Wymagania

- JDK 17 (zgodnie z konfiguracją `pom.xml`),
- Apache Maven 3.x.

8.2 Kompilacja

```
mvn -f graph/pom.xml clean package
```

Po kompilacji klasy znajdują się w `graph/target/classes`.

8.3 Uruchamianie GUI (start z Main)

Aplikacja GUI uruchamiana jest przez klasę `pl.graph.Main`, która inicjuje warstwę Swing.

Uruchomienie z linii poleceń:

```
java -cp graph/target/classes pl.graph.Main
```

Uruchomienie z IDE:

- konfiguracja Run/Debug: `pl.graph.Main`,
- working directory: katalog główny projektu.